

1. Objetivo

Este relatório apresenta um resumo da análise de manutenção de software realizada no projeto aep_back usando o SonarCloud. O objetivo é registrar a situação geral do código, identificar os principais problemas apontados pela ferramenta e propor ações de correção para melhorar qualidade, confiabilidade, segurança e manutenibilidade. A análise considera os resultados visíveis nos prints enviados. Portanto, este documento não é uma exportação automática da ferramenta; ele resume e interpreta os dados exibidos no painel do SonarCloud.

2. Evidência da análise

A tela de resumo mostra que o Quality Gate ficou como “Not computed”, porque as mudanças recentes tinham menos de 20 linhas e algumas verificações foram ignoradas pela ferramenta. Mesmo assim, a análise apresentou métricas úteis sobre segurança, confiabilidade, duplicação e manutenibilidade.

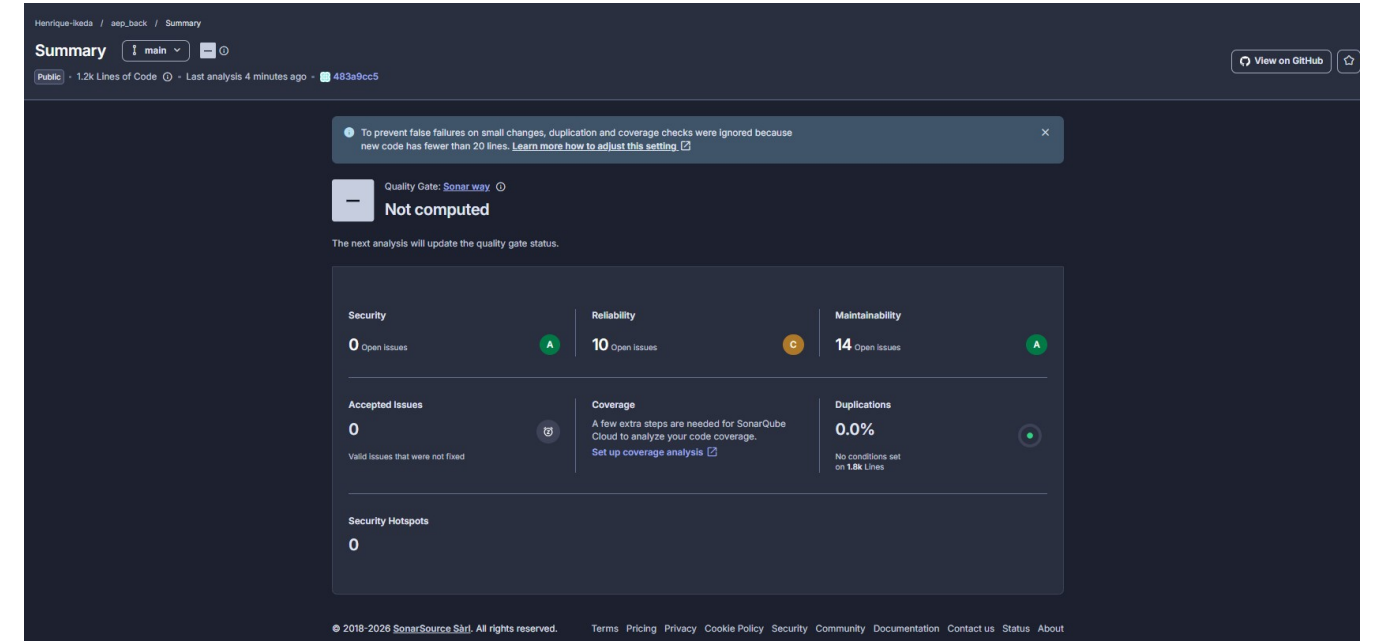


Figura 1 - Resumo do projeto no SonarCloud.

3. Resultados gerais da qualidade do código

Os indicadores principais demonstram que o projeto não possui problemas abertos de segurança nem pontos sensíveis de segurança, além de apresentar duplicação de código em 0,0%. O ponto mais crítico está em confiabilidade, que recebeu classificação C, com 10 issues abertas. A manutenibilidade recebeu classificação A, porém ainda existem 14 issues abertas classificadas principalmente como code smells.

Indicador	Resultado observado	Interpretação
Security	0 issues abertas; rating A	Não foram identificados problemas diretos de segurança no código analisado.
Security Hotspots	0 hotspots	Não houve pontos que exigissem revisão manual de segurança.
Reliability	10 issues abertas; rating C	Existem problemas que podem afetar comportamento correto, estabilidade ou previsibilidade.
Maintainability	14 issues abertas; rating A	O código é considerado mantível, mas ainda possui débito técnico a ser reduzido.
Duplications	0,0%	Não há evidência relevante de trechos duplicados.
Coverage	Sem dados configurados	A cobertura de testes não foi informada; é necessário configurar coleta de cobertura.
Accepted Issues	0	Nenhum problema foi marcado como aceito ou ignorado manualmente.

4. Leitura dos resultados

- Pontos positivos: ausência de issues de segurança, ausência de security hotspots e duplicação em 0,0%.
- Ponto de atenção principal: confiabilidade com rating C, indicando que parte das correções deve ser priorizada para evitar comportamento incorreto em produção.
- Ponto secundário: 14 issues de manutenibilidade, que não impedem o funcionamento imediato, mas dificultam evolução e leitura do código.
- Limitação da análise: a cobertura de testes não foi apresentada. Assim, não é possível avaliar quanto do projeto está protegido por testes automatizados.

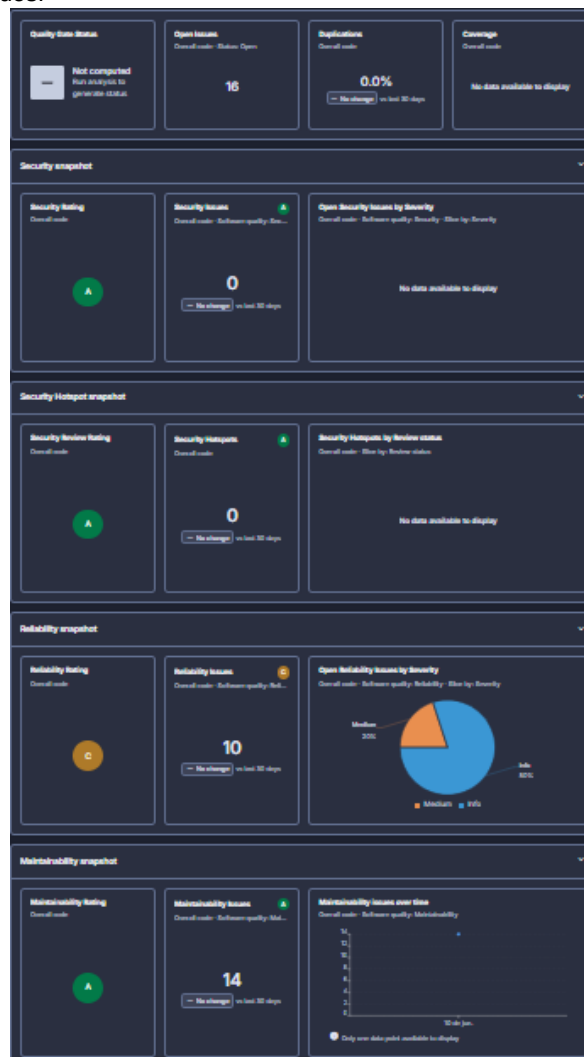


Figura 2 - Snapshot com ratings e distribuição de issues por categoria.

5. Issues encontradas

A lista de issues mostra recomendações de melhoria no código-fonte. Entre os problemas observados estão uso de saída direta no console, ausência de timezone explícito em chamadas de data/hora, duplicação de literais, métodos não utilizados e trechos que podem ser modernizados.

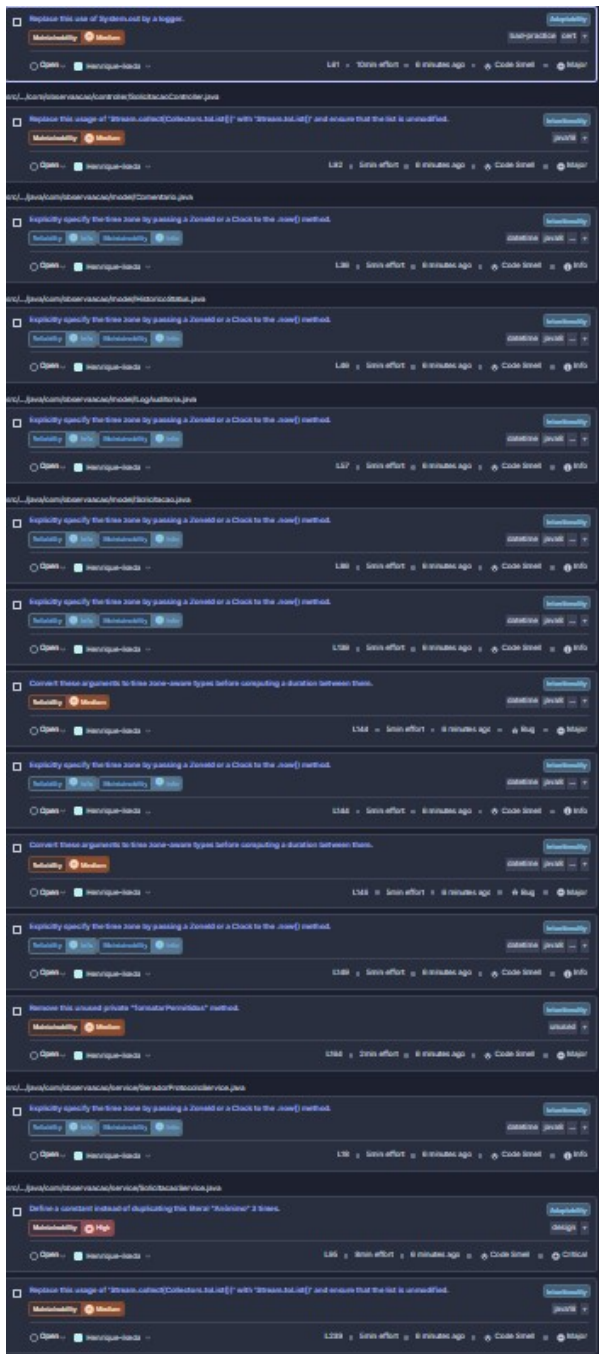


Figura 3 - Lista principal de issues no SonarCloud.

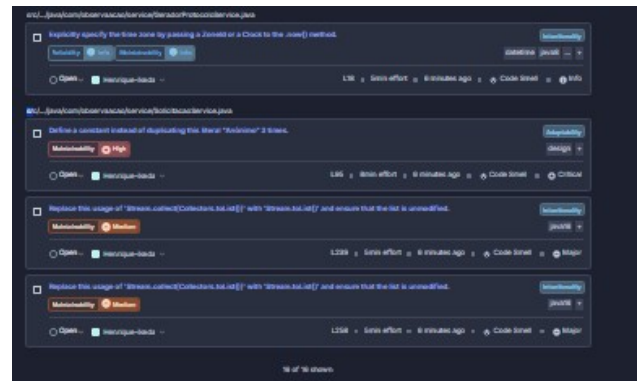


Figura 4 - Continuação da lista de issues.

Principais exemplos identificados:

- Substituir `System.out` por logger, melhorando rastreabilidade e controle de logs.
- Informar timezone explicitamente em chamadas de data/hora, como `now()`, para reduzir risco de erro em ambientes diferentes.
- Substituir `Stream.collect(Collectors.toList())` por `Stream.toList()`, quando aplicável.
- Remover método privado não utilizado, reduzindo código morto.
- Criar constante para literal repetido, como "Atendente", evitando duplicação e facilitando manutenção.

6. Impacto das issues

As issues de confiabilidade devem ser tratadas primeiro porque podem provocar resultados inconsistentes, principalmente quando envolvem manipulação de data, hora e timezone. As issues de manutenibilidade têm impacto mais gradual: elas tornam o código mais difícil de alterar, testar e revisar, aumentando o risco de erro em futuras evoluções do sistema.

7. Arquitetura e distribuição de responsabilidades

Na tela de arquitetura, o SonarCloud aponta 1 ocorrência de “Oversized” e 7 ocorrências de “Split responsibilities”. Isso indica que existem partes do projeto que podem concentrar responsabilidades demais ou estar divididas de forma pouco clara. Não foram identificados tangles ou weak tangles, o que é positivo para a estrutura geral.

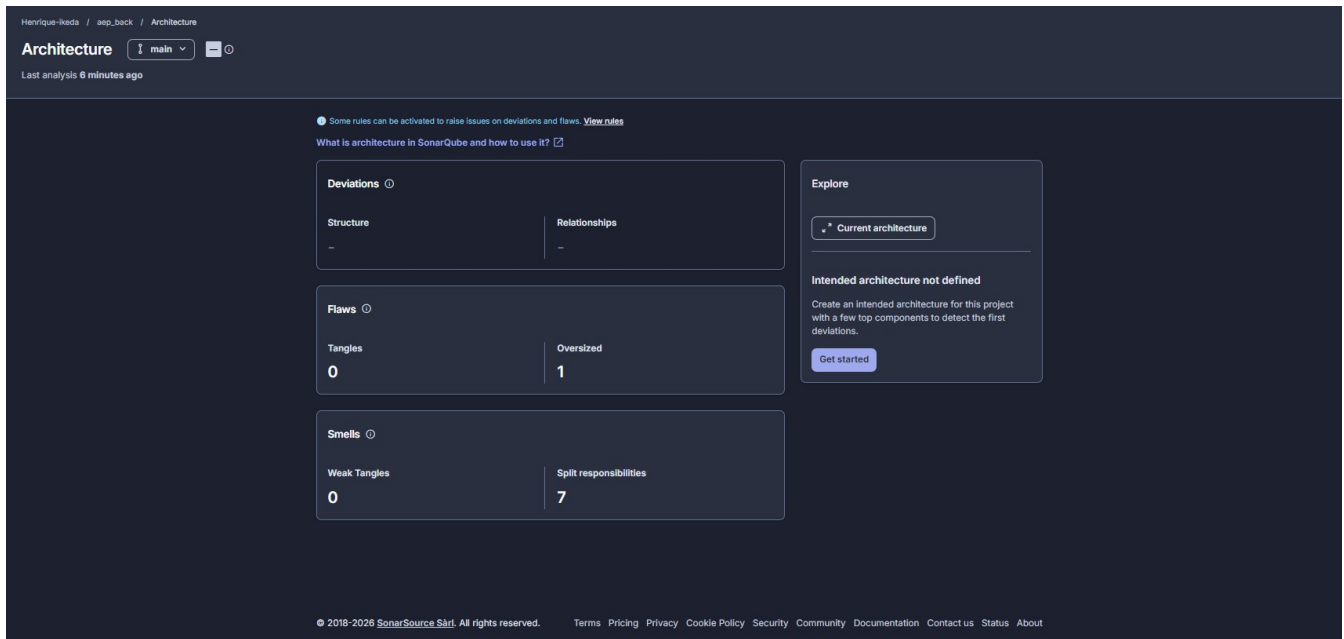


Figura 5 - Visão de arquitetura do projeto no SonarCloud.

8. Recomendações de melhoria

1. Corrigir primeiro as 10 issues de confiabilidade, especialmente regras relacionadas a timezone e comportamento previsível do sistema.
2. Configurar cobertura de testes no pipeline ou na execução local, permitindo que o SonarCloud apresente dados de coverage.
3. Substituir impressões em console por uma biblioteca de logging adequada.
4. Remover código morto e extrair constantes para valores repetidos.
5. Refatorar classes ou métodos muito grandes e revisar trechos com responsabilidades divididas.
6. Rodar novamente a análise após as correções para comparar a evolução dos indicadores.

9. Conclusão

De forma geral, o código analisado apresenta bons sinais em segurança e duplicação, pois não foram encontrados problemas abertos nessas categorias. Porém, a qualidade ainda pode melhorar: a confiabilidade está classificada como C, com 10 issues abertas, e a manutenibilidade possui 14 issues que representam débito técnico. A prioridade deve ser corrigir as issues que podem afetar o funcionamento do sistema e, em seguida, reduzir code smells para facilitar futuras alterações.

Após essas correções, recomenda-se executar uma nova análise no SonarCloud e incluir cobertura de testes para obter um diagnóstico mais completo da qualidade do projeto.